

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

دا كذا مرجع لاي حد داخل مثلا انترفيو او بيحاول يراجع ع EF مراجعة سريعة

Object Relation Mapping

اولا ايه هي ORM؟

Entity Framwork :: Object-Relational Mapping (ORM)

ORM"هو إطار عمل من مايكروسوفت ببسّهل التعامل مع قواعد البيانات من غير ما أكتب SQL بشكل مباشر.

بدل ما أتعامل مع ال Tables وال columns ، بقدر أتعامل مع البيانات كـ Classes و (Objects) في C# .

يعني مثلاً Table في قاعدة البيانات بيتحول عندي لـ Classe ، Columns بيتحول Properties ف الكلاس ، وبكده بقدر أقرأ وأحدث البيانات بكود C# عادي .

معلومة مهمة: الـ EF Core هي Cross-Platform مما يعني أنها تعمل على Mac, Windows, Linux ، وتدعم قواعد بيانات متعددة مثل SQL Server, MySQL, PostgreSQL وحتى SQLite.

يعني ببساطة:

- الـ Class في الكود ↔ Table في قاعدة البيانات
- الـ Properties في الكلاس ↔ Column في Table
- (object) ↔ (row)

Age	Name	Id
20	Ahmed	1
22	Salma	2

🔗 لما تستخدم ORM هيحصل الآتي:

1. الجدول **Students**

يتحول إلى **Class** ↔ في الكود:

```
public class Student {  
    public int Id { get; set; }  
    public string Name { get; set; }  
    public int Age { get; set; }  
}
```

2. الأعمدة **(Columns)** **Id, Name, Age**

تتحول إلى **(Properties)** ↔ في الكلاس:

- Id** ↔ **public int Id**
- Name** ↔ **public string Name**
- Age** ↔ **public int Age**

3. الصف **(Row)** الواحد في الجدول

يتحول إلى **(Object)** من الكلاس **Student:**

```
var student = new Student {  
    Id = 1,  
    Name = "Ahmed",  
    Age = 20  
};
```



✓ليه نستخدم ORM ؟

- أسهل وأسرع في الكتابة || بيخلي الكود كله بـ **C#** من غير ما تكتب **SQL** كتير
- بينظم التعامل مع **database** || بيقل الأخطاء

1. إيه الفرق بين **Code First** و **Database First** ؟

- **Code First** : تبدأ تكتب الـ **classes** في الكود، و **Entity Framework** بينشئ الـ **Tables** في **database**
- **Database First** : تبدأ من **database** موجودة، و **EF** بيولد منها الكلاسات (**models**) .

بصى يسيدى لعمل إيه مشروع **EF Core** بسيط محتاج تعمل **download** لبعض الـ **Packages**

& تثبيت المكتبات (**Nuget Packages**)

المكتبة الأساسية للتعامل مع **>> SQL Server** #

Install-Package >> Microsoft.EntityFrameworkCore.SqlServer

مكتبة الأدوات التي تتيح لنا استخدام أوامر المايكریشن **>> (Add-Migration, etc)** #

Install-Package >> Microsoft.EntityFrameworkCore.Tools

ملاحظة اذا كنت تستخدم **DotNet Cli** الـ (**Terminal**) الاوامر تختلف هنا ، **dotnet** **(add package)** لكن النتيجة واحده.

DbContext

: هو الكلاس الرئيسى ف **Entity Framework** للتعامل مع الـ **database**

هو اللي بيخليك تقدر تشتغل على الـ **database** كأنها **Object** ف **C#** بدل ما تكتب كود **Sql** بنفسك



ApplicationDbContext <=== ده كدا حلقة الوصل بين ال الابلكشن بتاعك وبين الداتا بيزا او هو ده الكلاس اللي ال **Entity Framework Core** هتتعامل معه يعنى هو ده الكلاس اللي هيضم ال **References** بتاعت كل ال **Classes** اللي هتمثل **Tables** فالداتا بيزا

➤ خطوات الانشاء :

- (1) انشاء كلاس (**Application dbContext**)
- (2) بعد كدا نخليه يورث من **DbContext**
- (3) نحدد **Connection String** داخل دالة **.OnConfiguring**.

```
public class ApplicationDbContext : DbContext
{
    // دالة الإعدادات: هنا نحدد نوع الداتابيز ومسارها

    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        // تعريف ال SQL Server استخدام ال Connection String
        optionsBuilder.UseSqlServer("Data Source=.;InitialCatalog=EFCore_Db;Integrated Security=True");
    }
}
```

توضيح: في المشاريع الحقيقية ال (**ASP.NET Core**) لا نكتب ال **Connection String** هنا بل نضعه في **appsettings.json** ونمرره عبر ال **Constructor (Dependency Injection)** لكن في ال **Console App** للتبسيط

◆ عندي ثلاث طرق هقدر اعرف بيهم **Entity Framework**

انى الكلاس ده عبارته عن جدول (**Domain Model**) فى الداتا بيزا



```

1- public DbSet<Blog> Blogs { get; set; }
2- protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Blog>();
}
3- public List<post> posts { get; set; }

```

بتع العلاقات

* أمر `Add-Migration MigrationName <<< Add-Migration`

Migration هو أمر يقوم بمقارنة الحالة الحالية للكود (**Models**) مع آخر **Snapshot** محفوظ ل **database**، ثم يولد ملف **Migration** يحتوي على التغييرات المطلوبة.

هذه التغييرات تكون مكتوبة على شكل **C# code** وليس **SQL** مباشر،

• أمر `Update-Database`

Update-Database هو أمر يقوم بأخذ ملفات **Migration** التي تم توليدها ثم يحولها إلى أوامر **SQL** ويقوم بتنفيذها فعليًا على **database** (السيرفر).

• تمام، خُلينا نوضح ملف الـ **Migration** وملف الـ **Snapshot** بشكل بسيط ومترتب 📁

1- ملف **Migration** :::

وهي عبارة عن **class** بداخله **function 2** واحدة اسمها **up** سيكون فيها **Update** اللى هتحصل على الداتا بيز بعد ما غيرت ف **Classes** بتاعتك والتانيه اسمها **Down** لو عملت جدول وحبيت بعد كذا ايجي امسحه فهنفذها وظيفتها التراجع (**Rollback**) لو أردنا إلغاء الـ **Migration**

```
protected override void Up(MigrationBuilder migrationBuilder)
{
    // يكون فيها Updates التي تحصل ع الداتا بيز بعد ما غيرت في classes بتعتك
}

protected override void Down(MigrationBuilder migrationBuilder)
{
    // دا التي هيقا فيها drop + بتستخدم لما يكون عايز تلغي التغيرات التي حصلت ف Up
}

```

(2) ملف **ModelSnapshot** :: هذا الملف يمثل "حالة الداتا بيز الحالية" في نظر الكود.

⚠ تحذير هام جداً:

لا تقم أبداً بحذف ملف الـ **ModelSnapshot** أو التعديل عليه يدوياً الـ **EF Core** تعتمد عليه لمعرفة الفرق بين الكود الجديد و الداتا بيز القديمة.

في حالة حذفه، ستظن **EF Core** أن قاعدة البيانات غير موجودة، وستحاول إنشاء الجداول من الصفر، مما يؤدي إلى حدوث أخطاء مثل: (Table already exists)

- **EF Core** ممتاز في إنشاء الجداول، لكن العمليات المتقدمة (**View / SP / Trigger**) تغيير نوع عمود قد تحتاج **Migration** مخصصة أو **SQL** يدوي.
- كيفية حذف أو التراجع عن **Migration**
- ⚠ خطأ شائع جداً: إذا قمت بإنشاء **Migration** واكتشفت فيها خطأ، لا تقم أبداً بحذف ملف الـ **Migration** يدوياً (**Delete Key**) خصوصاً إذا كنت قد طبقتها على **database** لأن ذلك سيؤدي إلى عدم التزامن بين الكود و **database**

- الطريقة الصحيحة للتراجع
- تتم العملية على خطوتين:
- إرجاع قاعدة البيانات للوراء
- يجب أولاً إلغاء تأثير الـ **Migration** من السيرفر باستخدام الأمر:

<اسم المايكریشن السابقة> **Update-Database**

مثال : إذا كانت آخر **Migration** هي **AddSales** والتي قبلها **Initial** نكتب : **Update-Database Initial**
حذف ملف الكود

بعد أن تصبح قاعدة البيانات نظيفة ومتزامنة،
يمكننا حذف ملف الـ **Migration** بأمان باستخدام الأمر:

Remove-Migration

❖ **update-database -migration:0** : ارجع لقاعدة البيانات لحالتها الأصلية قبل ما تكون عملت أي تغييرات (أي قبل ما تستخدم **migration**)

✅ **لو Migration** لسه ما اتطبقتش على **Database** هتقول **Remove-Migration**

❖ لحذف **Migration** معينه **Update-database (MigrationName)**

💡 انترفيو الفرق بين **.NET Core** & **.NET Framework** ::

❖ **.NET Framework** :

- بدأ من زمان (2002) || بيشتغل بس على **Windows** || مش مفتوح المصدر
- مناسب للتطبيقات القديمة (**Legacy apps**) .

❖ **.NET Core** :

- ظهر سنة 2016 || مفتوح المصدر || أداء أسرع



- بيشتغل على **Windows و Linux و Mac**
- تقدر تنشر التطبيق في أكثر من نظام (Cross-platform)

❖ يعني إيه "مقفول المصدر" (Closed Source) ؟

البرمجيات المقفولة المصدر هي برامج أو منصات ما بتقدرش تشوف الكود البرمجي بتاعها، لأن الشركة اللي طورتها بتحافظ عليه كملكيّة خاصة. يعني الكود مش متاح للناس إنها تطلع عليه أو تعدّل فيه.

EFMigrationsHistor

Table دا بيكون فيه **Raw** لكل **migration** عملتها **update** على الداتا بيز بتاعتي

❖ مثلاً انا عملت **Class** جديد اسمه **Product** مثلاً وعايظه يسمع في الداتا بيز **add-Migration** وبعد كذا عملت **Class** تاني اسمه **Product** وسمعتة ف الداتا بيز.

نفترض اني عايز امسح جدول ال **Product** دا هعمل اي. همسح الكلاس من **Model** وهمسح **DbSet** بتاعته من ال **DbContext** كذا انا مسحته من الابلكيشن متبقي امسحه من الداتابيز ودا بيتم اني بروح اعمل **add-migration** جديدة وهو بيروح يقارن يشوف اي التغير اللى حصل وبيعمله ابديت في الداتابيز

```

1-DataAnnotations =====>
[Required] ==> NULL  دا يعني بقوله اني العمود مطلوب يعني مينفعش يكون ب

2-fluantApi =====>
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Blog>()
        .Property(b => b.Id)
        .IsRequired();
}

```




[unicode(false)] <==== NotUnicode خلى فى بالك ده هو هو كانه بقوله انه ده

اما لو قالى انى العمود هيبقى [Column(TypeName ="Nvarchar(100)")] هو هو لو قالى انه unicode

unicode | Nvarchar <====> يعنى دعم جميع الاحرف واللغات والرموز ولكنه يستهلك مساحة اكبر ف قاعدة البيانات
unicode(false)] | NotUnicode <====> دعم اللغة الانجليزية فقط ويقلل من حجم البيانات

- لو كتبت "Hello" باستخدام VARCHAR كل حرف بيأخذ 1 بايت
- لو كتبت "Hello" باستخدام NVARCHAR كل حرف بيأخذ 2 بايت
- ببساطة NVARCHAR بيستهلك مساحة أكبر لأنه بيخزن كل حرف في 2 بايت عشان يدعم كل اللغات
- ببساطة VARCHAR بيستهلك مساحة اقل لأنه بيخزن كل حرف في 1 بايت عشان يدعم اللغة الانجليزية فقط



1-DataAnnotations <====>

[Key]

public int Id { get; set; } Primary key هيقا Column انه هذا

2-fluantApi <====>

```
modelBuilder.Entity<Employee>( )  
.HasKey(e => e.Id);
```



1-DataAnnotations <====>

[NotMapped] { Attribute } دا لو عايز اشيل اى عمود من جدول بحط فوقه

2-fluantApi<====>

```
protected override void OnModelCreating(ModelBuilder modelBuilder) لازم الجدول كله  
{  
    modelBuilder.Ignore<post>();  
}
```

```
modelBuilder.Entity<Employee>( ) لازالة عمود  
.Ignore(e => e.Name);
```



لو عايز اغير اسم الجدول

1-DataAnnotations ==>

[Table("posts")] بحط هذه الكلمة فوق الجدول

2-fluantApi ==>

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<post>( )
        .ToTable("zeko");
}
```

Schema لو عايز اغير اسم

1-DataAnnotations ==>

[Table("posts",Schema ="blogng")] بحط هذه الكلمة فوق الجدول

2-FluantApi ==>

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<post>( )
        .ToTable("posts", schema: "bbb");

    modelBuilder.HasDefaultSchema("bbb"); default هو تبيقا ده لاسكيميا ده
}
```





1-DataAnnotations ==>

```
[primaryKey(nameof(Url),nameof(addone))]
```

2-fluantApi ==>

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
```

```
{
```

```
    modelBuilder.Entity<post>()
```

```
    .HasKey(e => new {e.name,e.author})
```

ده لو عندي Compset primary key يعني عندي عمودين primary key مينفعوش يتكررو مع بعض خالص

```
}
```



1-DataAnnotations ==>

```
[Column(TypeName = "varchar(100)")]    لو عايز تغير نوع الداتا طيب
```

```
public string Name { get; set; }
```

```
[Column(TypeName = "double")]
```

```
public int price { get; set; }
```

2-fluantApi ==>

```
modelBuilder.Entity<Employee>(e =>
```

```
{
```

```
    e.Property(e => e.Name)
```

```
    .HasColumnType("varchar(100)");
```

```
    e.Property(e => e.id)
```

```
    .HasColumnType("varchar(100)");
```

```
});
```





1-DataAnnotations ==>

ده لو عايز اضيف تعليق

[Comment("the primarey key employee")] ده لو عايز اضيف تعليق

```
public int Id { get; set; }
```

2-fluantApi ==>

```
modelBuilder.Entity<Employee>( )  
    .Property(e => e.Id)  
    .HasComment("mohamed reda ");
```



1-DataAnnotations ==> default value ده لو عندك قيمه وعازي تخليها

```
[DefaultValue(2)]
```

```
public string Url { get; set; }
```

2-fluantApi ==>

```
protected override void OnModelCreating(ModelBuilder modelBuilder)  
{  
    modelBuilder.Entity<Blog>( ) default value ده لو عندك قيمه وعازي تخليها  
        .Property(e => e.Ratung)  
        .HasDefaultValue(2);  
}
```



1-DataAnnotations ==>

لو عايز تغيير اسم عمود داخل الجدول

```
[Column("URL")]
```

```
public string Url { get; set; }
```

2-fluantApi ==>

```
modelBuilder.Entity<Post>()
```

```
.Property(e => e.Tital)
```

```
.HasColumnName("ttt");
```



fluantApi ==>

بس خلى بالك ده يعتبر مش Column حقيقى يعنى اى Virtual Column ==

```
protected override void OnModelCreating (ModelBuilder modelBuilder)
```

```
{
```

```
    modelBuilder.Entity<Author>()
```

```
    .Property(b => b.DisplayName)
```

```
    .HasComputedColumnSql("[LastName] + ',' + [firstName]");
```

لو عايز تدمج قيمتين مع بعض

```
}
```

```
var _cont = new Application();
```

```
var author = _cont.authors.Find(1);
```

Update على قيمه

```
author.LastName = "Hatem"
```



خلى بالك انا لو خليت Id اى نوع غير Int وليكن مثلا byte مش هيقى identity فغلشان كدا استخدمت هذا الكود

1-DataAnnotations =====>

```
[DatabaseGenerated(DatabaseGeneratedOption.Identity)]
```

```
public byte Id { get; set; }
```

2-fluantApi =====>

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
```

```
{
```

```
    modelBuilder.Entity<Categori>()
```

```
        .Property(c => c.Id)
```

```
        .ValueGeneratedOnAdd(); id ----> identity خلى
```

```
}
```

1. Cascade لو حذف ال (Parent)، يتم حذف ال (Child) تلقائياً.

مثال : عند حذف "Order"، يتم حذف كل "OrderItems" التابعة له تلقائياً

2. SetNull :: لو حذف ال (Parent)، يتم تعيين القيمة في العمود المرتبط في ال (Child) إلى Null

مثال: لو حذف "Teacher"، حقل TeacherId في جدول "Students" يتحول إلى Null

3. Restrict :: يمنع حذف ال (Parent) طالما فيه Childs مرتبطين بيه.

مثال: مش هتقدر تحذف "Category" لو فيه "Products" مرتبطة بيه.

Scaffold-DbContext :: هي أداة في EF Core بتسمح لي إني أعمل Reverse Engineering لقاعدة بيانات موجودة، وأولد منها الكلاسات و DbContext تلقائياً.

✓ معناها Reverse Engineering :

"أستخدم قاعدة البيانات علشان أولّد منها كود (Models + DbContext) C# تلقائياً".



ده كدا لو انا عايزه يستخدم **DataAnnotations** وليس **fluantApi**

Scaffold-DbContext "Data Source=.;Initial Catalog=BikeStores;Integrated Security=True;TrustServerCertificate=True"

Microsoft.EntityFrameworkCore.SqlServer -**OutputDir** Models -**ContextDir** Data -**DataAnnotation**

Scaffold-DbContext "Data Source=.;Initial Catalog=BikeStores;Integrated Security=True;TrustServerCertificate=True"

Microsoft.EntityFrameworkCore.SqlServer -**OutputDir** Models -**ContextDir** Data

* ده لما يكون عايز استدعى جداول معين من الداتا بيز

```
protect potentially sensitive information in your connection string, you should move it out of source code. You can avoid scaffolding read it from configuration - see https://go.microsoft.com/fwlink/?linkid=2131148. For more guidance on storing connection strings, see scaffold-dbContext 'Data Source=(localdb)\ProjectsV13;Initial Catalog=EFCore' Microsoft.EntityFrameworkCore.SqlServer -Tables Blogs, Output Package Manager Console Web Publish Activity
```

◆ **Index** هو اداة او طريقة تستخدم لتسريع عمليات البحث والاستعلامات على قاعدة البيانات، خاصة تلك التي تشمل البحث أو الفلترة أو ترتيب البيانات.

لماذا نستخدم الـ **Index** ؟

\$ تحسين الأداء: يساعد المؤشر في تسريع عمليات البحث أو التصفية أو ترتيب البيانات في الأعمدة المستخدمة بشكل متكرر.

\$ تقليل زمن الاستجابة: من خلال تحسين أداء الاستعلامات، يمكن أن يقلل من الوقت الذي يستغرقه تنفيذ الاستعلامات.

\$ تحسين العمليات التي تحتوي على **JOIN** أو **WHERE**



[كيف تؤثر الفهارس (Indexes) على الأداء]

:: Index

في قاعدة البيانات تعمل بطريقة مشابهة للفهرس في الكتاب: تساعدك في الوصول إلى البيانات بسرعة
عندما تقوم بإنشاء فهرس على عمود في قاعدة البيانات، فإنه يُسهل البحث عن السجلات بناءً على هذا العمود،
ولكن كلما كان هناك المزيد من الفهارس، كلما أصبح من الصعب على قاعدة البيانات تعديل البيانات بسرعة

[ما الذي يحدث عند استخدام DELETE و UPDATE و INSERT ؟]

1. عندما تجري هذه العمليات على قاعدة البيانات (مثل إضافة بيانات جديدة أو تحديثها أو حذفها)، يجب على قاعدة البيانات أن تُحدث الفهارس الخاصة بهذه البيانات أيضاً
2. هذا يعني أن الفهرس يجب أن يُعدل كلما أضيفت أو حدثت أو حذفت بيانات، وهذا يستغرق وقتاً إضافياً.

(1) عملية INSERT (إضافة بيانات جديدة) :

* عندما تضيف ريكورد جديداً إلى الجدول، تقوم قاعدة البيانات بإضافة هذا ال ريكورد إلى index أيضاً

* إذا كان لديك عدة فهرس على الأعمدة، يجب على قاعدة البيانات إضافة ال ريكورد الجديد إلى جميع هذه الفهارس

* إذا كان لديك الكثير من الفهارس، فهذا سيستغرق وقتاً أطول لأن قاعدة البيانات يجب أن تحدث جميع الفهارس



(2) عملية UPDATE (تحديث بيانات) :

إذا قمت بتحديث قيمة عمود تم فهرسته، يجب على قاعدة البيانات تحديث الفهرس ليتناسب مع القيمة الجديدة

* إذا كان هناك عدة فهرس على هذا العمود، فسيستغرق الأمر وقتًا أكبر لأن قاعدة البيانات يجب أن تحدث جميع الفهارس المرتبطة بالعمود الذي قمت بتحديثه

(3) عملية DELETE (حذف بيانات) :

عندما تقوم بحذف ريكورد من قاعدة البيانات، يجب على قاعدة البيانات أيضًا حذف ال ريكورد من **Index**

إذا كانت لديك عدة فهرس، يجب أن تقوم قاعدة البيانات بحذف ال ريكورد من جميع الفهارس المرتبطة بالجدول

[انواع Index]

- Single Column Index
- Composite Index

نعمل Index

1-DataAnnotations ==>

[index(nameof(اسم العمود))] بحظه فوق الجدول

2-fluantApi ==>

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Post>( )
        .HasIndex(v => v.Tital);
}
```





ده لو عايز اعمل **Compset Index** باستخدام **(DataAnnotations)**

```
[Index(nameof(FirstName),nameof(LastName))]  
  
protected override void OnModelCreating(ModelBuilder modelBuilder)  
{  
    modelBuilder.Entity<Person>() fluantApi ده لو عايز اعمل Compset Index باستخدام  
    .HasIndex(p => new {p.FirstName, p.LastName});  
}
```



لعمل **Unique**

1-DataAnnotations ==>>

[Index(nameof(FirstName),IsUnique =true)] بحطه فوق الجدول

2-fluantApi ==>>

```
protected override void OnModelCreating(ModelBuilder modelBuilder)  
{  
    modelBuilder.Entity<Person>( )  
        .HasIndex(p => p.FirstName)  
        .IsUnique();  
}
```

DataAnnotations ==>> Index لو عايز تغير اسم

```
[Index(nameof(LastName),Name ="Index_PP")]
```

fluantApi ==>>>>> Index لو عايز تغير اسم

```
protected override void OnModelCreating(ModelBuilder modelBuilder)  
{  
    modelBuilder.Entity<Person>( )  
        .HasIndex(p => p.FirstName)  
        .HasDatabaseName("index_lastname_test");  
}
```



Script-Migration  هي أداة تساعدك على تحويل التغييرات التي أجريتها على

model إلى كود SQL يمكنك مراجعته أو تطبيقه يدويًا على قاعدة البيانات
لماذا نستخدمها؟

* مراجعة التعديلات قبل تطبيقها: يمكنك أن ترى بالضبط ما سيتم تغييره في قاعدة البيانات
التنفيذ اليدوي: في بعض الحالات قد لا ترغب في تطبيق التغييرات تلقائيًا، فإمكانك استخدام
Script-Migration لتنفيذ التغييرات يدويًا

```
Script-Migration 1:5 <----- طب لو عايز اعمل Script من 2 : 5 تعمل اى
Script-Migration 5:9 <----- طب لو من من 6-->9
Script-Migration 4 <----- طب لو من من 5 لآخر حاجة
Remove-Migration -Name MigrationName إذا كنت تريد حذف migration محددة بالاسم، يمكنك استخدام
```

```
Date_Seeding لو عايز تضيف داتا تكون موجود فى الجدول علشان تتأكد انى الجداول شغاله اما لا
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Blog>() ( ) دا لما يكون جدول واحد
        .HasData(new Blog { BlogId =1,Url="www.google.com"});
    modelBuilder.Entity<Post>() Foreign Key علىه
        .HasData(new Post { BlogId = 1, PostId = 1, Tital = "Post1", Content = "Test1" },
        new Post { BlogId = 1, PostId = 2, Tital = "Post2", Content = "Test2" } ) ;
}
modelBuilder.Entity<Blog>()
    .HasData(new Blog { Id = 1, Url = "www.google.com" },
        new Blog { Id = 2, Url = "www.youtube.com" },
        new Blog { Id = 3, Url = "www.microsoft.com" },
        new Blog { Id = 4, Url = "www.udomey.com" });
```



(لو عايز اعمل **select** لكل الداتا بيز اللي عندي في الجدول)

```
ApplicationDbContext _context = new ApplicationDbContext();  
var blogs = _context.posts.ToList();  
foreach (var item in blogs)  
Console.WriteLine(item.Content);
```

طب لو عايز اعمل **Select** ل **Row** معين

```
ApplicationDbContext _context = new ApplicationDbContext();  
var Blogs = _context.posts.Find(1);  
Console.WriteLine($"ID: {Blogs.PostId}: {Blogs.Content}");
```

First باستخدامها لوانا عارف اني في الجدول فيه **Row** واحد على الاقل ومنطبق عليه هذا الشرط اللي بداخله

```
ApplicationDbContext _context = new ApplicationDbContext();  
var Blogs = _context.posts.First(m=>m.PostId>1);  
Console.WriteLine($"ID: {Blogs.PostId}: {Blogs.Content}");
```



لو انا مثلا لو معنديش **row** ده اخليها **FirstOrDefault** علشان ميطلعش اي ايرور

```
ApplicationDbContext _context = new ApplicationDbContext();  
var Blogs = _context.posts.FirstOrDefault(m=>m.PostId>6);  
Console.WriteLine(Blogs==null? "No Items found": $"ID: {Blogs.PostId}: {Blogs.Content}");
```

الفرق بين **First** && **FirstOrDefault**

```
Int [] number = [] ;
```

First لو راحت ملكتش ولا **Element** موجود هترمي **Exception**

FirstOrDefault لو راحت ملكتش ولا **Element** موجود هترجع **default value** اللي هو الصفر





اي الفرق بين المثال الاول والثاني

```
1) Var result = Cars
```

```
.Where (C=>C.Make)
```

```
.Last();
```

```
2) Var result = Cars
```

```
.Last(C=>C.Make);
```

Where يمشى من الاول للاخر لحد ما يلاقى اللي ينطبق ع الشرط

Last يمشى من الاخر للاول لحد ما يلاقى اللي ينطبق ع الشرط

فهنا Last افضل من where علشان performance



بستخدامها لو عايز اجيب اخر عنصر فى الجدول بس لازم استخدم معها

OrderBy ==> Last لاسترجاع آخر عنصر بالترتيب المحدد

```
ApplicationDbContext _context = new ApplicationDbContext();
```

```
var Blogs = _context.posts.OrderBy(m => m.PostId).Last();
```

```
Console.WriteLine(Blogs==null? "No Items found": $"ID: {Blogs.PostId}: {Blogs.Content}");
```

لو انا مثلا لو معديش raw ده اخليها LastOrDefault علشان ميطلعش اى ايرور

```
ApplicationDbContext _context = new ApplicationDbContext();
```

```
var Blogs = _context.posts.OrderBy(m => m.PostId).LastOrDefault(m=>m.PostId>5);
```

```
Console.WriteLine(Blogs==null? "No Items found": $"ID: {Blogs.PostId}: {Blogs.Content}");
```

Single: يتم استخدام طريقة **Single()** لإرجاع العنصر الوحيد من المجموعة، والذي يلبي الشرط. في حالة العثور على أكثر من عنصر واحد في المجموعة أو عدم العثور على أي عنصر في المجموعة، فسوف يتم طرح خطأ **InvalidOperationException**



ما هو الفرق بين طرق `First()` و `Single()`

`First()` يبحث في مجموعة البيانات ويُرجع أول عنصر يتوافق مع الشرط المحدد. إذا كانت المجموعة فارغة (أي لا تحتوي على أي عناصر)، فإن `First()` سيرمي خطأ من نوع `InvalidOperationException`. يمكن استخدامه عندما تتوقع وجود عنصر واحد على الأقل أو تريد العمل مع أول عنصر موجود في المجموعة.

`Single()` يقوم بالبحث في مجموعة البيانات ويُرجع عنصر واحد فقط. إذا كانت المجموعة تحتوي على أكثر من عنصر واحد يتوافق مع الشرط المحدد، سيرمي خطأ من نوع `InvalidOperationException` أيضًا. إذا كانت المجموعة فارغة، أيضًا سيرمي استثناء. يستخدم `Single()` عندما تتوقع أن يكون هناك عنصر واحد فقط في المجموعة يتوافق مع الشرط.

متى تستخدم كل منهما؟

استخدم `First()` إذا كنت لا تهتم بعدد العناصر المتوافقة مع الشرط وتريد أول عنصر فقط. استخدم `Single()` عندما تتأكد من وجود عنصر واحد فقط يتوافق مع الشرط في المجموعة. طب لخص الفرق بين `SingleOrDefault` و `FirstOrDefault` و `LastOrDefault`

1) `SingleOrDefault()`

- المهمة: يبحث عن عنصر واحد فقط يطابق الشرط.
- إذا كانت المجموعة فارغة، ستعيد القيمة الافتراضية (default) لنوع البيانات المعين
- إذا كان العنصر من نوع `int`، سيتم إرجاع 0.



- إذا كان العنصر من نوع **Reference** (مثل كائنات)، سيتم إرجاع **Null** إذا كان هناك أكثر من عنصر يطابق الشرط: يرمى استثناء من نوع **InvalidOperationException**.
- متى تستخدم؟ : عندما تتوقع أن يكون هناك عنصر واحد فقط يتوافق مع الشرط.

2) **LastOrDefault()** .

- المهمة: يبحث عن آخر عنصر يطابق الشرط.
- إذا كانت المجموعة فارغة: يُرجع القيمة الافتراضية.
- إذا كان هناك أكثر من عنصر يطابق الشرط: يُرجع آخر عنصر فقط يطابق الشرط.

FirstOrDefault():

- المهمة: يبحث عن أول عنصر يطابق الشرط.
- إذا كانت المجموعة فارغة: يُرجع القيمة الافتراضية.
- إذا كان هناك أكثر من عنصر يطابق الشرط: يُرجع أول عنصر فقط يطابق الشرط.
- متى تستخدم؟ : عندما تحتاج إلى أول عنصر يتوافق مع الشرط أو إذا كنت تريد قيمة افتراضية إذا لم يوجد أي عنصر.

var Blogs = _context.Products

OrderBy(m => m.ProductName).

ThenBy(m => m.ProductId).

ThenByDescending(m => m.ModelYear).ToList();()

OrderBy(m => m.ProductId) : ترتيب البيانات أولاً حسب اسم المنتج (**ProductName**) بشكل تصاعدي (الترتيب الافتراضي)

ThenBy(m => m.ProductName) : إذا كان هناك منتجات بنفس الاسم، يتم ترتيبها ثانياً حسب **ProductName** بشكل تصاعدي.

ThenByDescending(m => m.ModelYear) : إذا كانت هناك منتجات بنفس الاسم و **ProductId**، يتم ترتيبها ثالثاً حسب **ModelYear** بشكل تنازلي



كل الناس اللي بتبدء بحرف Z

```
ApplicationDbContext _context = new ApplicationDbContext();  
var Blogs = _context.posts.Where(m=>m.Name.StartsWith("z").ToList());  
foreach (var item in Blogs)  
    Console.WriteLine(item.Content);  
  
ApplicationDbContext _context = new ApplicationDbContext();  
var Blogs = _context.posts.Where(m=>m.Id>500).ToList();  
foreach (var item in Blogs)  
    Console.WriteLine(item.Id);
```

Select

```
ApplicationDbContext _context = new ApplicationDbContext();  
var Blogs = _context.posts.Select(m=> new {PostId=m.PostId,Tital=m.Tital}).ToList();  
foreach (var item in Blogs)  
    Console.WriteLine($"ID: {item.PostId}:{item.Tital}");
```

Distinct

```
ApplicationDbContext _context = new ApplicationDbContext();  
var Blogs = _context.posts.Select(m => new { cont = m.Content, Tital = m.Tital }).Distinct().ToList();  
foreach (var item in Blogs)  
    Console.WriteLine($"ID: {item.cont}:{item.Tital}");
```

هنا بقوله ابدء من 11 الى 20 Skip && Take

```
ApplicationDbContext _context = new ApplicationDbContext();  
var Blogs = _context.posts.Skip(11).Take(20).ToList();  
foreach (var item in Blogs)  
    Console.WriteLine(item);
```



ما هو الفرق بين **Take** و **Skip** في LINQ ؟

• إجابة نموذجية:

- **Take(n)** يعيد أول **n** عنصر من المجموعة.
- **Skip(n)** يتجاهل أول **n** عنصر ويعيد العناصر المتبقية بعدهم.

```
Sum

ApplicationDbContext _context = new ApplicationDbContext();
var Blogs = _context.posts.Sum(m => m.Salary);
Console.WriteLine(Blogs);

====

ApplicationDbContext _context = new ApplicationDbContext();
var Blogs = _context.posts.Where(m => m.PostId > 5).Sum(m => m.Salary);
Console.WriteLine(Blogs);

Max

ApplicationDbContext _context = new ApplicationDbContext();
var Blogs = _context.posts.Max(m => m.PostId);
Console.WriteLine(Blogs);

Min

ApplicationDbContext _context = new ApplicationDbContext();
var Blogs = _context.posts.Min(m => m.PostId);
Console.WriteLine(Blogs);

OrderBy

ApplicationDbContext _context = new ApplicationDbContext();
var Blogs = _context.posts.OrderBy(m => m.PostId).ToList();
foreach (var item in Blogs)
    Console.WriteLine($"ID: {item.PostId}:{item.Tital}:{item.Content}" );

OrderByDescending

ApplicationDbContext _context = new ApplicationDbContext();
var Blogs = _context.posts.OrderByDescending(m => m.PostId).ToList();
foreach (var item in Blogs)
    Console.WriteLine($"ID: {item.PostId}:{item.Tital}:{item.Content}" );
```



Average

```
ApplicationDbContext _context = new ApplicationDbContext();  
var Blogs = _context.posts.Average(m => m.Total);  
Console.WriteLine(Blogs);  
  
ApplicationDbContext _context = new ApplicationDbContext();  
var Blogs = _context.posts.Where(m => m.PostId > 5).Average(m => m.Total);  
Console.WriteLine(Blogs);  
  
Count = int 32  
ApplicationDbContext _context = new ApplicationDbContext();  
var Blogs = _context.posts.Count();  
Console.WriteLine(Blogs);  
  
ApplicationDbContext _context = new ApplicationDbContext();  
var Blogs = _context.posts.Count(m => m.PostId > 1);  
Console.WriteLine(Blogs);
```

Count = int 64

```
ApplicationDbContext _context = new ApplicationDbContext();  
var Blogs = _context.posts.LongCount();  
Console.WriteLine(Blogs);  
  
ApplicationDbContext _context = new ApplicationDbContext();  
var Blogs = _context.posts.LongCount(m => m.PostId > 1);  
Console.WriteLine(Blogs);  
  
CountBy==== (CountBy) <==== باستخدام ProductName لكل العناصر عدد العناصر  
ApplicationDbContext _context = new ApplicationDbContext();  
var product = _context.Products.CountBy(c => c.ProductName);  
foreach (var item in product)  
{ Console.WriteLine(value: $"{item.Key}---> {item.Value}");}  
- (ProductName التي تم تجميع البيانات عليها) هنا تكون item.Key  
- item.Value عدد المنتجات التي لديها نفس قيمة ProductName
```





Max

```
ApplicationDbContext _context = new ApplicationDbContext();  
var Blogs = _context.posts.Max(m => m.PostId);  
Console.WriteLine(Blogs);
```

Min

```
ApplicationDbContext _context = new ApplicationDbContext();  
var Blogs = _context.posts.Min(m => m.PostId);  
Console.WriteLine(Blogs);
```



دا كذا لو عايز تضيف فى الجدول بس فى اخر list

```
ApplicationDbContext _context = new ApplicationDbContext();  
var Blogs = _context.posts.Where(m => m.PostId > 500).ToList().Append(new Post { BlogId=2,PostId  
=3,Tital = "post3",Content ="Test3"});  
foreach (var item in Blogs)  
    Console.WriteLine($"Id:{item.PostId}:{item.Tital}:{item.Content}");
```

دا كذا لو عايز تضيف فى الجدول بس فى الاول list

```
ApplicationDbContext _context = new ApplicationDbContext();  
var Blogs = _context.posts.Where(m => m.PostId > 0).ToList().Prepend(new Post { BlogId=1,PostId  
=4,Tital = "post4",Content ="Test4"});  
  
foreach (var item in Blogs)  
    Console.WriteLine($"Id:{item.PostId}:{item.Tital}:{item.Content}");
```





Any ده باستخدامها لو انا عندي Raw ف الجدول هيرجع True ملقيش هيرجع False

```
ApplicationDbContext _context = new ApplicationDbContext();  
var Blogs = _context.posts.Any();  
Console.WriteLine(Blogs);
```

All ده باستخدامها لو انا في الشرط اللي يطبقه ده في Raw واحد بس مطبقش عليه الشرط هيديني False يعني لازم كل Raw يطبق عليه الشرط علشان يدي True

```
ApplicationDbContext _context = new ApplicationDbContext();  
var Blogs = _context.posts.All(m=>m.PostId>1);  
Console.WriteLine(Blogs);
```

كيف يمكنك استخدام Any و All في LINQ ؟

• إجابة نموذجية:

○ Any تستخدم للتحقق من وجود عنصر واحد أو أكثر يطابق الشرط.

○ All: تستخدم للتحقق من أن جميع العناصر تطابق الشرط.

DataAnnotation هي عبارة عن مجموعة من الاتريبوتس بنستخدمهم علشان نعمل data validation

وبتكون موجودة جوا namespace اسمها DataAnnotation

* هي مش بتسمع ف الداتا بيز هي بتكون على الانبوت اللي انا بضيف في الاسم

[DatabaseGenerated(DatabaseGeneratedOption.None)] يعني متعملش Identity



```
var context = new ApplicationDbContext(); // لاضافة داتا  
var employee = new Employee { Name = "Ahmed" };  
context.employees.Add(employee);  
context.SaveChanges();
```



قبل الدخول في أنواع العلاقات ، يجب أن نفهم مصطلحاً يتكرر كثيراً وهو

Navigation Property

Navigational Propertie دول هما الخصائص اللي بتسمحك تنتقل بين (Entities) المرتبطة في ال Database. يعني، بدل ما تعمل Joins يدوية زي في SQL العادي، EF بيخليك تكتب كود أبسط وأقرب لل OOP

في البداية، Entity Framework هو (Object-Relational Mapping) ORM بيربط بين كودك الـ #C و ال Database. والـ Navigational Properties هي جزء أساسي من ده، زي روابط افتراضية (Virtual Properties) في الكلاسات اللي بتمثل الجداول.

مثال بسيط: لو عندك كلاس Student وكلاس Course ، وكل طالب ممكن يكون له كورسات كتير (One-to-Many) ، هتكون الـ Navigational Property زي Courses <Course> ICollection في كلاس Student. ده بيخليك تقدر تقول student.Courses وتوصل لكل الكورسات اللي تابعة للطالب ده.

1. إزاي بتعمل الـ Navigational Properties ؟

الـ Navigational Properties مش بس خصائص عادية؛ هي بتعتمد على (Relationships) اللي بتحددها في ال Model في EF ، بتقدر تعرفها بطريقتين:

- **Convention** : الـ EF بيْفهم العلاقات تلقائياً لو سميت الخصائص صح (زي StudentId كـ (Foreign Key)
- **Fluent API** : لو عايز تخصيص أكثر، تستخدم OnModelCreating في ال DbContext عشان تحدد الـ HasOne / HasMany / WithOne / WithMany



```

public class Student {
    public int Id { get; set; }
    public string Name { get; set; }
    public virtual ICollection<Course> Courses { get; set; } // Navigational Property }

public class Course {
    public int Id { get; set; }
    public string Title { get; set; }
    public int StudentId { get; set; } // Foreign Key
    public virtual Student Student { get; set; } // Navigational Property العكسية
}

```

هنا، **Courses** هي الـ **Navigational Property** للوصول من الطالب للكورسات، و **Student** للوصول العكسي. الكلمة **virtual** مهمة هنا عشان تفعل الـ **(Lazy Loading)** تحميل البيانات لاحقاً لما تحتاجها.

2. أهمية وجود الـ **Navigational Properties** في الـ **Queries**

الـ **Navigational Properties** هي السر في كتابة **Queries** فعالة وسهلة في **EF** بدونها، هتضطر تعمل كل حاجة يدوياً، وده بيأثر على الأداء والكود.

• لو موجودة (الفوائد):

◦ تسهيل الـ **LINQ Queries**: تقدر تكتب `=> context.Students.Include(s => s.Courses.Where(s => s.Name == "Ahmed"))`

والكورسات معاه في استعلام واحد (**Eager Loading**). ده بيمنع مشكلة **N+1 Queries** (يعني استعلام رئيسي + استعلام لكل عنصر مرتبط)

• **Lazy Loading**: لو مش عايز تحمل كل حاجة مرة واحدة، **EF** بيحمل البيانات تلقائياً لما تقول `student.Courses.Count()`. ده بيوفر موارد لو البيانات كبيرة.

• **Explicit Loading**: تقدر تتحكم في التحميل يدوياً بـ `context.Entry(student).Collection(s => s.Courses).Load()`.

- تحسين الأداء : بتقلل عدد الـ **Round Trips** لقاعدة البيانات، وبتخليك تستخدم **Projection** زي (**Select**) عشان تجيب بس اللي تحتاجه.
- دعم العلاقات المعقدة : زي **Many-to-Many** ، حيث **EF** بيعمل جدول وسيط تلقائيًا ويخليك تنقل بسهولة.

```
var students = context.Students
    .Include(s => s.Courses) // Eager Loading
    .ToList();

foreach (var student in students) {
    Console.WriteLine(student.Name + " has " + student.Courses.Count + " courses.");
}
```

العلاقات بين الجداول

```
[ one to one ]

public class Blog
{
    public int Id { get; set; }
    [Required][MaxLength(100)]
    public string Url { get; set; }
    public BlogImage BlogImage { get; set; }
}

public class BlogImage
{
    public int Id { get; set; }
    public string Image { get; set; }
    [Required] [MaxLength(100)]
    public string Caption { get; set; }
    public int BlogId { get; set; }
    [ForeignKey("BlogId ")]
    public Blog Blog { get; set; }
}
```



[One to One]

1-DataAnnotations =====> ForeignKey Entity framework مش بتقدر تحدد في بعض الاحيان بيقولك

[ForeignKey("العمود")] ForeignKey بحطها فوق العمود اللي هو هيكون

[ForeignKey("BlogId ")]

```
public Blog Blog { get; set; }
```

2-fluantApi =====>

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
```

```
{
```

```
    modelBuilder.Entity<Blog>() --parent
```

```
    .HasOne(b => b.BlogImage) --من اليمين
```

```
    .WithOne(l => l.Blog) --من الشمال
```

```
    .HasForeignKey<BlogImage>(l => l.BlogId);
```

```
}
```

```
[ many to many ]
```

```
public DbSet<post> posts { get; set; }
```

```
public DbSet<Tag> tags { get; set; }
```

```
public class post
```

```
{
```

```
    public int Id { get; set; }
```

```
    public string Tital { get; set; }
```

```
    public string Content { get; set; }
```

```
    public ICollection<Tag> tags { get; set; }
```

```
}
```

```
public class Tag
```

```
{
```

```
    public int TagId { get; set; }
```

```
    public ICollection<post> posts { get; set; }
```

```
}
```



[One To Meny]

```

[ One To Meny ]

public class Blog // [ Parent ] OR [ Principal Entity ] ممكن اسميه
{
    public int Id { get; set; } // [ Principal Key(Pk) ]
    public string Url { get; set; }
    public List<post> posts { get; set; } // Collection Navigation Property
}

public class post // [ Chaild ] OR [ Dependent Entity ] ممكن اسميه
{
    public int Id { get; set; }
    public string Tital { get; set; }
    public string Content { get; set; }
    public int BlogId { get; set; }
    [ForeignKey("CourseId")]
    public Blog Blog { get; set; } // Reference Navigation Property
    // Reference: تعني أنها تشير إلى كائن آخر في الذاكرة (أو جدول آخر في قاعدة البيانات)
}

```

[MaxLength(50)] لتحديد أقصى length للفيلد
[StringLength(50, MinimumLength = 10)] لتحديد أقصى length وأقل length للفيلد
[Column(TypeName = "money")] مع فيلدات الساليري أو الفلوس
[Range(22, 60)] لتحديد رينج لانتجر فيلد زي فيلد ال age مثلا
[EmailAddress] من خلالها تقدر تحدد ان الفيلد دا ميقبالش غير ايميل وكمان ممكن استخدم الداتا تايب اللي فوق مع المييل (الاسكرين)
[Phone] مع ارقام الفون وممكن استخدم الداتا تايب اللي ف الاسكرين .
[NotMapped] لو انا عايز اعمل بروبرتي ميكونش ليها تمثيل في الداتا بيز (يعني اعمل بروبرتي بس ميعملهاش كفيلد في الداتابيز) .
[InverseProperty("Employees")] لتحديد ال navigation دي خاصة ب انهبي نافيجشن بروبرتي في الجدول التاني.
[RegularExpression("^([0-9]{1,2})-[a-zA-Z]{5-20}\$" , ErrorMessage = "must be like 12-stCairo")]





[Chaild]OR[Parent] سوء في [FluentApi==> One To Meny] وموجود Navigation Property

```
public class Blog
{
    public int Id { get; set; }
    public string Url { get; set; }
    public List<post> posts { get; set; }    // Collection Navigation Property
}

public class post
{
    public int Id { get; set; }
    public string Tital { get; set; }
    public string Content { get; set; }
    public int BlogId { get; set; }
    public Blog Blog { get; set; }          // referance Navigation Property
}

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Blog>()
        .HasMany(b => b.posts)
        .Withone(s => s.Blog);
}
```





[Child] OR [Parent] سوء في Navigation Property ومش موجود FluentApi ==> One To Many في حالة العلاقة

```
public class Blog // parent
{
    public int Id { get; set; }
    public string Url { get; set; }
}

public class post // Child
{
    public int Id { get; set; }
    public string Title { get; set; }
    public string Content { get; set; }
    public int BlogId { get; set; }
}

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Blog>() // parent
        .HasMany<post>()
        .WithOne()
        .HasForeignKey( p=>p.BlogId);
}
```



```
public class Blog
{
    public int Id { get; set; }
    public string Url { get; set; }
    public List<post> posts { get; set; } post يحتوي ع العديد من
}

// ده معناه ان Blog يحتوي ع العديد من post
```



طبعاً عارفين أن EF ال default بتاعها أنها شغاله Tracking معناها أن أي تعديل أو أي عملية بتعملها على الداتا بيز ال EF بتفضل تعمل لها Tracking وبالتالي أول ما تعمل (SaveChange) كل التغييرات دي بتتخفظ في الداتا بيز - فلو أنت بتعمل select للداتا علشان تعدل عليها فهي في الحالة دي لازم تكون Tracking علشان التغيير ده يسمع في الداتا بيز - ولو بتعمل select للداتا علشان تعرضها فقط ف الأفضل أنها تكون No Tracking بتكون أفضل في ال performance

في ال EF7 حصل تحديث جديد ببحسن ال Performance وهو ال Delete Bulk Update عرفنا أن EF هي عبارة عن Tracking ف أي تغيير أو تحديث بمجرد ما تعمل (SaveChange) بيسمع في الداتا بيز - افترض أنك بتعمل التغيير ده سواء (Insert , Update , Delete) على مجموعة كبيرة من الداتا بيز ال هيكون Performance بطيء جداً - ف ال EF عملت حاجه جديده بتحسن الاداء وهي لو أنت بتعمل تعديل على 1000 Row مثلاً ال EF بتقسم ال Command ده وتبعثها الداتا بيز مثلاً نفترض 10 مرات في كل مره تبعث 100 Row فقط يحصل عليهم التحديث ف بدل ما تبعث 1000 Request للداتا بيز هتبعث فقط 10 Request في كل Request موجود فيه 100 - Row ولاكن تظل النقطة الأفضل أن التعديل /التحديث بيتم على مستوي الداتا بيز نفسه مش بيتم على مستوي Application وده اكيد احسن في ال Performance

```
return context.Users.FirstOrDefault(u => u.Id == userId) ; Tracking
```

```
return context.Users.AsNoTracking().FirstOrDefault(u => u.Id == userId); nottracking
```

*ال Default behavior بتاع ال EFCore انه بيعمل Tracking ولو عايز اقله متعملش tracking هقله AsNoTracking طيب

امتا اعمل tacking لو انا محتاج اعمل اضافة او تعديل او حذف على الكود بتاعى فهو بيراقب الكود وبيقول حالته

*وامتا اعمل AsNoTracking في حالة اني بعمل سليكت على موظف معين وانا عايز ارجعه فقط ومش عايز اعمل عليه تعديلات زي الابديت او حذف او اضافة عشان لو انا سايبه على الديفولت هو بيروح يترك الكود فبعمل خطوة زيادة انه بيتراك الكود وانا مش عايز ودا بيأثر على performance بتاعت الكود بتاعى



* خلى فى بالك انت فى ال **AsNoTracking** لو عوزت تعمل اى ابدیت او حذف او اضافة مش هينفع لان **AsNoTracking** مش بتشتغل غير لما يكون عايز اسليكت على حاجة معينه او ارجعه فقط

```
AsNoTracking ده كدا لو عايز اخلى الديفلت بتاعى بيقا  
context.ChangeTracker.QueryTrackingBehavior = QueryTrackingBehavior.NoTracking;  
var result = context.Customers.FirstOrDefault(e => e.CustomerId == 2);  
result.FirstName = "AAAAAAA";  
context.SaveChanges();
```

استراتيجيات تحميل البيانات المرتبطة (Loading Strategies)

هذا الموضوع هو سبب 90 % من مشاكل الاداء

Eager Loading in Entity Framework

EAGER LOADING

بس هنا هيقابلني exception ليه بقي ؟
لان ف اللحظة ال عملنا فيها selection ل books ال author
متعملوش selection

طيب اي الحل قالك اننا نستخدم ال eager loading ازاي عن طريق
include method

```
static void Main()  
{  
    var context = new ApplicationsDbContext();  
    var book = context.Books.Include(i => i.Author).Single(e => e.Id == 1);  
    Console.WriteLine(book.Author.Name);  
}
```

EAGER LOADING

تعال نشرح ع مثال يعني اي ال eager loading

```
static void Main()  
{  
    var context = new ApplicationsDbContext();  
    var book = context.Books.Single(e => e.Id == 1);  
    Console.WriteLine(book.Author.Name);  
}
```

هنا عندنا كلاس ال books فيها navigation property من كلاس author طيب
اي ال انا عايزه هنا عايز اعرض اسم مؤلف الكتاب ال id بتاعه يساوي 1

EAGER LOADING

السطر بتاع include ده عمل اي بقي ؟
راج عمل selection ل books and author وبقيت خلاص اقدر
استخدمهم واجيب اسم المؤلف
وطبعا اقدر اعمل اكتر من include لكذا table عادي جدا
ومن هنا نطلع بتعريف ال eager loading بقى انه بيحمل الداتا كلها
من الداتا بيز مره واحده يعني لو عامل include ل table ثالث من
مستخدمه برضه هيروح يعمل selection من db علشان كده ال eager
loading بيأثر ع ال performance

✗ تبعد عنه امتى؟

-لو ال Child عنده data كثير وأنت مش محتاجها كلها.

-لو هتعمل Includes كثير ال Query هتبقى معقدة وتغرقك JOINS .

⚠ بس خلي بالك لازم تستخدم ال ThenInclude لو عايز كل الداتا بتاعت ال Child بتاع

ال Child علشان ال Include لوحدها مش بتجيب غير ال Data بتاعت ال Child اللي انت طالبها
بس.

LAZY LOADING

هنا بقي خلاص بقيت اقدر استخدم ال lazy loading عادي من غير
include او اي حاجه

```
static void Main()
{
    var context = new ApplicationsDbContext();
    var book = context.Books.Single(e => e.Id == 1);
    Console.WriteLine(book.Author.Name);
}
```

LAZY LOADING

قالك ال lazy loading بقى انه بيحمل الداتا بيز ال هحتاجها بس
وده طبعا حل مشكله ال performance بتاعت ال eager loading
طيب ازاي استخدم ال lazy loading قالك اسهل طريقه اننا نروح
ننزل package اسمها
microsoft.EntityFrameworkCore.Proxies
بعد م تعمل installation ل ال package
بروح ف سطر ال connection واقوله ()UseLazyLoadingProxies()

```
protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
=> optionsBuilder.UseLazyLoadingProxies().UseSqlServer("Data Source=LAPTOP-
```

✓ امتى تنفع Lazy Loading ؟

-لما ال Data تكون كثير ومش عايز تجيبها كلها مره واحده.

-لما UI يكون محتاج ال Data على أجزاء.

✗ تبعد عنها امتى؟

-لو هتستخدمها جوا... loop هيحصل N+1 Problem.

الكارثة: (N+1 Problem)

- دي أهم نقطة: لو عندك 100 طالب وبتعمل عليهم loop عشان تطبع كورساتهم، الـ **Lazy Loading** هيروح للداتا بيز 101 مرة! (مرة يجيب الـ 100 طالب، و 100 مرة عشان يجيب كورسات كل واحد لوحده) ده بيدمر الأداء تماماً.



```
virtual foreign key هذا column التي هذا  
public virtual Book Book { get; set; }  
public virtual Author Author { get; set; }
```

LAZY LOADING



نيجي بقي للجزء المهم هل بما ان ال lazy loading بيحل مشكله ال performance بتاع eager loading اننا منستخدروش ؟
طبعا الاجابه لا
ساعات هتحتاج ال eager loading لو كانت العلاقات بين ال table معقده ف هتبقى محتاج تحمل الداتا كلها مره واحده
ف الخلاصه : الاختيار بين Lazy Loading و Eager Loading بيعتمد على نوع التطبيق والحاجات. لو معظم الوقت بتحتاج كل البيانات، Eager Loading أحسن. بس لو بتحتاج بيانات محددة بس، Lazy Loading هو الأمثل.

ESLAM AYMAN

طب انهى افضل فيهم؟

بص يا صديقي، سؤالك "أنهى أفضل" ملوش إجابة واحدة مطلقة لأنها بتعتمد على السيناريو اللي أنت شغال فيه، بس القاعدة العامة في الشغل الاحترافي بتميل للـ **Eager Loading** (باستخدام Include)

خليني أخص لك تختار مين فيهم بناءً على المعلومات اللي معانا:

1. الـ **Eager Loading** (الأفضل في أغلب الحالات)
هو الاختيار الأول للمبرمجين حالياً لعدة أسباب:
 - الأداء المتوقع: أنت بتبعت طلب واحد للداتا بيز وبيرجع بكل حاجة، فبتبقى عارف إن الـ **Query** دي هتاخد وقت كذا وخلاص.
 - تجنب الكوارث: ببيحميك من مشكلة الـ N+1 اللي بتبوظ الأداء تماماً في الـ **Lazy Loading**
 - متى تستخدمه؟ لما تكون متأكد إنك محتاج الـ (**Child Data**) مثلاً بتعرض جدول فيه اسم الطالب وجنبه أسماء كورساته.
2. الـ **Lazy Loading** (استخدمه بحذر شديد)

- الميزة: بيوافر داتا لو أنت مش متأكد إن المستخدم هيفتح التفاصيل ولا لأ.
- العيب: بيعمل "طلبات خفية" للداتا بيز؛ ممكن تلاقي الأبلكيشن بطيء وأنت مش عارفليه، وتكتشف إنه بيعت 100 طلب صغير بدل طلب واحد كبير.
- متى تستخدمه؟ في شاشات الـ **Admin** المعقدة اللي فيها علاقات كتير جداً والمستخدم نادراً لما بيحتاج يشوف "كل" التفاصيل المرتبطة مرة واحدة.

3: Explicit Loading

- ده بقى زي الـ **lazy** بس اعقل شوية
- ما بيعملش أي **Query** لوحده... يعني لما تنادي على الـ **navigation property** مش هيجيب الـ **data** من الـ **database** لا انت محتاج تحدد اذا كانت **Reference** او **Collection** و بعد كذا تعمل انت **load** بنفسك.
- و ممكن كمان نستخدم الـ **Where** لو عايزين نفلتر الـ **children** ✓
 -ليه ممكن تحتاج **Explicit Loading** ؟
 -لما تكون عايز تتحكم إمتى الـ **data** تتجاب.
 -عايز تجيب **child** بشرط معين.
 -تبعد عنه امتي؟ ✗
- لو هتستخدمه جوه **loop** نفس مصيبة **Lazy** .
- لو عايز كل حاجة تتجاب مرة واحدة : روح **Eager** وخلص.

2. الفرق بين الـ **Reference** والـ **Collection**

- زي ما وضحت، لازم تحدد نوع العلاقة وأنت بتعمل الـ: **Load**
- **Reference** : بتستخدمها لو العلاقة "واحد" مثلاً: طالب ليه عنوان واحد
 (**Student.Address**).
- **Collection** : بتستخدمها لو العلاقة "كتير" (مثلاً: طالب ليه كورسات كتير
 (**Student.Courses**))

```
// (فلتر) Where مع استخدام Collection لـ Explicit Loading
context.Entry(student)
    .Collection(s => s.Courses)
    .Query() // هنا بنفتح الـ Query
    .Where(c => c.Grade > 50)
    .Load(); // هنا بقى الداتا بتيجي فعلاً من الداتا بيز
```


شرح الكود خطوة بخطوة:

Context.Entry(student) : أنت تقول للـ **EF** : بص على الطالب ده اللي معاك في الـ **Memory** دلوقتي.

Collection(s => s.Courses) : أنا عايزك تركز على قائمة الكورسات الـ **(Collection)** المرتبطة بالطالب ده.

Query : (هنا المرتبط الفرس) دي معناها: "يا **EF** ، متجيبش الكورسات كلها فوراً.. استنى! أنا لسه هكتبلك شروط **(Query)** تطبقها في الداتا بيز قبل ما تبعثلي الداتا." .

من غير **Query()** ، كنت هتضطر تحمل كل الكورسات وبعدين تفلترهم في الـ **C#** وده بيستهلك ذاكرة.

Where(c => c.Grade > 50) : دي بقى الفلتر اللي هتحصل جوه الداتا بيز يعني الداتا بيز هتدور فقط على الكورسات اللي درجتها أكبر من 50 وتجهزهم.

Load() : دي "أمر التنفيذ" بتقول للـ **EF** يلا ابعت الـ **SQL** اللي فات ده للداتا بيز وهاتلي النتيجة حطها جوه اوبجكت الطالب اللي معايا."

ليه الحركة دي جامدة جداً؟

- تخيل إن الطالب عنده 100 كورس، بس هو سقط في 90 ونجح في 10:
- في الـ **Lazy Loading** كان هيحمل الـ 100 كورس كلهم غصب عنك أول ما تلمس كلمة **Courses**.
 - في الـ **Explicit Loading** مع **Query** : الداتا بيز هتبعثلك الـ 10 كورسات اللي نجح فيهم بس.
- النتيجة :أنت وفرت "زحمة" في الداتا اللي منقولة من الداتا بيز للأبليكيشن بتاعك.

Single Queries

يدور أسلوب **Single queries** في **EF Core** حول الحصول على جميع البيانات المطلوبة دفعة واحدة. إنه يشبه الحصول بفارغ الصبر على كل ما نحتاجه في استعلام قاعدة بيانات واحد بيبكون مفيداً عند التعامل مع نماذج البيانات المعقدة أو جلب البيانات ذات علاقه ببعضها لانه بياثر على **performance** فمش باستخدامه الا ف الجداول المعقدة عادةً، لا تؤثر البيانات المتكررة بشكل كبير على أداء التطبيق بل على الداتا بيز نفسها



```
var companies = _dbContext.Companies
    .Include(company => company.Departments)
    .ToList();
```

Split Queries

على عكس طبيعة **Single queries** المتكامل، تقوم **Split queries** بتقسيم عملية استرجاع البيانات إلى خطوات أصغر. يعد هذا الأسلوب مفيدًا عندما نريد جلب البيانات الضرورية فقط، وتجنب الإفراط في الجلب

```
var companies = _dbContext.Companies
    .Include(company => company.Departments)
    .AsSplitQuery()
    .ToList();
```

Use Split Queries by Default

```
protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
{
    optionsBuilder.UseSqlServer(
        "Server=(localdb)\\MSSQLLocalDB;Integrated Security=true;Initial
        Catalog=SingleAndSplitQueriesInEFCore",
        o => o.UseQuerySplittingBehavior(QuerySplittingBehavior.SplitQuery));
}

Main
var companies = _dbContext.Companies
    .Include(company => company.Departments)
    .AsSplitQuery()
    .ToList();
```



بيقولك طب افرض دالوقت انت اشتغلت بطريقة **Split Queries by Default** وعايز اطبق حته **Single queries** اعمل اى

```
var custom = _context.Customers
    .Include(b => b.Orders)
    .AsSingleQuery()
    .ToList();
```

باختصار، يمكن أن يكون **single querying** أكثر كفاءة في السيناريوهات التي نحتاج فيها إلى الكثير من البيانات ذات الصلة في وقت واحد

من ناحية أخرى، قد يكون **SplitQuery** مفيدًا عندما نريد تحسين حالات استخدام محددة أو تقليل كمية البيانات التي يتم جلبها في وقت معين
إذا كنا نحتاج في كثير من الأحيان إلى مجموعة كبيرة من البيانات المتصلة، فيجب علينا اختيار نهج **single querying**. إذا أردنا أن نكون انتقائيين ودقيقين، فإن الاستعلام **SplitQuery**.

الخلاصة :

1. الـ **Single Query** (كلنا في مركب واحدة)

دا الـ **Default** في الـ **Entity Framework** هو بيعت استعلام واحد للداتا بيز فيه **LEFT JOIN** لكل الجداول المرتبطة.

- إزاي بيشتغل؟ بيروح الداتا بيز ويجيب الطالب بكل كورساته بكل امتحاناته في "جدول واحد كبير" (**Result Set**).
- المشكلة الكبيرة (**Cartesian Explosion**): لو عندك طالب ليه 10 كورسات، وكل كورس ليه 10 امتحانات، الـ **Single Query** هيرجعلك **100 صف** !اسم الطالب هيتكرر 100 مرة في الداتا اللي راجعة.
- أثره على الأداء: زي ما قلت، بيضغط على الداتا بيز لأنها بتبني جدول عملاق، وبيضغط على الشبكة لأن فيه داتا متكررة كتير (**Redundant Data**) بتنقل "هواء" على الفاضي.

2. الـ **Split Query** (كل واحد يروح في تاكسي لوحده)



هنا الـ **EF** ييقسم الطلب لعدة استعلامات منفصلة (استعلام للطالب، واستعلام للكورسات، واستعلام للامتحانات) وبعدين يربطهم هو في الذاكرة.

- **الميزة الجبارة:** مفيش داتا متكررة. اسم الطالب هيرجع مرة واحدة، والـ 10 كورسات هيرجعوا في استعلام ثاني، والامتحانات في تالت.
- **ليه نستخدمه؟** لما يكون عندك **Includes** كتير (علاقات متشعبة)، الـ **Split Query** بيبقى أسرع بكتير لأن حجم الداتا اللي ماشية في الشبكة أقل بكتير.
- **العيب الخفي:** بما إنه بيعت أكثر من طلب، لو الداتا بيز عليها ضغط أو الشبكة بطيئة، "كترة المشاوير" ممكن تأخر الرد.

الحالة	الاختيار الأفضل	السبب
علاقة واحدة بسيطة (One-to-One)	Single Query	الـ Join بسيط ومش هيتكرر داتا كتير.
علاقات كتير (One-to-Many كتير)	Split Query	عشان تهرب من كارثة تكرار الداتا (Cartesian Explosion).
حجم الداتا اللي راجعة "عرض" (Columns) كتير	Split Query	عشان تقلل حجم الـ Result Set.

 التصدير إلى "جداول بيانات Google"

نصيحة المعلم: استخدم الـ **Single Query** طول ما الأمور ماشية تمام، وأول ما تلاقي الـ **Include** زاد عن 2 أو 3 وبدأت الدنيا تبطأ، أقلب فوراً لـ **Split Query**.

تحب أوريك إزاي بنخلي الـ **Split Query** هو الـ **Default** على مستوى الـ **Context** كله عشان متقعدهش تكتبه في كل حنة؟

```

BikeStoresContext context = new BikeStoresContext();
{
    List<Brand> brands = new List<Brand>
    {
        new Brand { BrandName = "Noor" }
    };

    context.Brands.AddRange(brands);
    context.SaveChanges();

    var brand = context.Brands.ToList();
    foreach (var item in brand)
    {
        Console.WriteLine(item.BrandName);
    }
}

```

لو عايز تعمل تعديل على جدول معين

```

BikeStoresContext context = new BikeStoresContext();

var category = new Category { CategoryId = 7, CategoryName = "moo" };

context.Update(category);

context.SaveChanges();

```

NULL بس هنا لو اعمده مش ميعوته مع الاعمده اللي عايز تعديل هيعتبرها كلها ب

هنا بعمل عادي تعديل وفي نفس الوقت بحافظ على الاعمده اللي مش مبعوته انها

متبقاش NULL بس خلى ف بالك هنا انا بعمل تعديل على عمود معين



```

var context = new BikeStoresContext();

var product = new Product{ProductId= 2,CategoryId=3,ProductName="SALEH"};

context.Update(product);

context.Entry(product).Property(p => p.CategoryId).IsModified = false;

context.Entry(product).Property(p => p.ModelYear).IsModified = false;

context.SaveChanges();

```

```

namespace EFECore
{
    0 references
    class Program
    {
        0 references
        static void Main(string[] args)
        {
            var _context = new ApplicationDbContext();

            var posts = _context.Posts.Where(p => !p.IsDeleted).ToList();

            foreach (var post in posts)
            {
                post.IsDeleted = true;
            }

            _context.UpdateRange(posts);

            _context.SaveChanges();
        }
    }
}

```

هنا بعمل تعديل ع الجدول كله

Remove لو عايز اعمل حذف لعمود معين

```

var cust = context.Products.Find(3);
context.Remove(cust);
context.SaveChanges();

```



Remove لو عايز اعمل حذف لاکثر من عمود

```
.Where(c => c.ProductId >= 5 && c.ProductId <= 9)
.ToList();
context.RemoveRange(cust);
context.SaveChanges();
```

وبس یسیدی دا کان مرجع لل EF یا رب یكون عجبکم

